



UT1. MANEJO DE FICHEROS

ACTIVIDAD 6

Lectura de XML con DOM

Título: Lectura de un documento XML de registros de seguridad con DOM en Java.

Descripción:

En esta actividad vamos a leer un fichero **XML de registros de seguridad** usando el parser DOM incluido en el JDK. El objetivo es practicar cómo cargar un archivo XML, recorrer su estructura y extraer información de cada elemento.

De esta forma aprenderás a trabajar con la estructura jerárquica de un XML y a acceder a datos concretos de cada registro, tal y como se hace en sistemas de **ciberseguridad** para analizar logs de accesos o eventos.

Objetivos:

- Comprender cómo utilizar el parser DOM para cargar y leer un fichero XML.
- Aprender a recorrer un árbol DOM y obtener datos de elementos y atributos.
- Practicar el uso de métodos como `getDocumentElement()`, `getElementsByTagName()` y `getAttribute()`.

Instrucciones:

1. Crea un nuevo proyecto Java en tu IDE y guárdalo con el nombre **LecturaXML_Seguridad**.
2. En el directorio del proyecto, guarda el fichero XML proporcionado: **logs.xml**.
3. Crea una nueva clase llamada **LeerLogs** dentro del proyecto.
4. En el método `main()`, desarrolla las siguientes operaciones:

Carga y analiza el archivo XML:

- Cargar el archivo `logs.xml` en un objeto `Document` utilizando `DocumentBuilderFactory` y `DocumentBuilder`.
- Obtener el elemento raíz del documento con `getDocumentElement()`.

Navegación del árbol DOM:

- Usar `getElementsByTagName("log")` para obtener una lista de registros.
- Para cada elemento `<log>`, dentro de un bucle, recorre los nodos de cada elemento `"log"` y extrae y muestra la información correspondiente en la consola:
 - El atributo **id**.
 - El valor de los elementos `<nivel>`, `<mensaje>` y `<usuario>`.

5. Maneja las excepciones necesarias para controlar posibles errores en la carga o lectura del XML.



Rúbrica:

0	No se cumple ninguno de los requisitos mínimos.
3	Presenta la tarea, pero el código no es correcto.
5	Se presenta la tarea y el código es correcto, pero no se demuestra su ejecución con evidencias (capturas o logs de consola).
7	Se presenta la tarea, el código es correcto, y se demuestra el funcionamiento mediante capturas o salida de consola; sin embargo, no se describen los principales métodos y clases del programa.
9	Se completa la tarea correctamente en todos sus apartados, con código funcional, capturas de consola y breve descripción de métodos y clases.
10	Se añade alguna funcionalidad extra o detalle innovador que mejore la aplicación (por ejemplo, mostrar estadísticas de cuántos registros INFO, WARNING y ERROR hay).

Entrega:

1. Incluye aquí el código fuente de la clase desarrollada (LeerLogs.java):

```
import java.io.File;
import java.io.IOException;
import javax.xml.parsers.DocumentBuilder;
import javax.xml.parsers.DocumentBuilderFactory;
import javax.xml.parsers.ParserConfigurationException;
import org.w3c.dom.Document;
import org.w3c.dom.Element;
import org.w3c.dom.NodeList;
import org.xml.sax.SAXException;

/**
 * @author Juan Gabriel Galarza Claros
 */

public class LeerLogs {

    static final String RUTA= "logs.xml";

    public static void main(String[] args) {
        try {
            // ---- PASO 1: cargar el XML en memoria ----
            // a) Crear la fábrica (DocumentBuilderFactory) y el constructor (DocumentBuilder).
            DocumentBuilderFactory fabrica = DocumentBuilderFactory.newInstance();
            DocumentBuilder analizador = fabrica.newDocumentBuilder();

            // b) Leer el fichero y convertirlo en un árbol DOM (Document).
            Document doc = analizador.parse(new File(RUTA));
            doc.getDocumentElement().normalize();

            // c) Obtener el elemento raíz y mostrar su nombre.
            System.out.println("Raíz: " + doc.getDocumentElement().getNodeName());
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```



```
// ---- PASO 2: recorrer los registros <log> ----
// a) Obtener la lista de todos los elementos <log>.
NodeList logs = doc.getElementsByTagName("log");

// b) Recorrer la lista con un bucle.
for (int i = 0; i < logs.getLength(); i++) {
    // c) Convertir cada nodo en Element para poder leer sus datos.
    Element u = (Element) logs.item(i);
    // d) Leer el atributo id.
    String id = u.getAttribute("id");

    // e) Leer el texto de <nivel>, <mensaje> y <usuario>.
    String nivel = u.getElementsByTagName("nivel")
        .item(0).getTextContent();
    String mensaje = u.getElementsByTagName("mensaje")
        .item(0).getTextContent();
    String usuario = u.getElementsByTagName("usuario")
        .item(0).getTextContent();

    // f) Mostrar los datos por consola.
    System.out.println(id + "; " + nivel + " (" + mensaje + ") + " (" + usuario + ")");
}

// ---- PASO 3: tratar las excepciones ----
// a) Error al configurar el parser.
// b) Error al abrir el fichero.
// c) El fichero no es un XML válido.
} catch (ParserConfigurationException | SAXException | IOException e) {
    System.err.println("Error al leer el XML: " + e.getMessage());
}
}

// ---- obtenerTexto(elemento, etiqueta) ----
// a) Buscar dentro del elemento las etiquetas con ese nombre.
// b) Si no hay ninguna, devolver "(sin dato)".
// c) Si la hay, devolver el texto de la primera.
public static String obtenerTexto( Element elemento, String etiqueta) {
    NodeList lista = elemento.getElementsByTagName(etiqueta);
    if (lista.getLength() == 0) {
        return "(sin dato)";
    }
    return lista.item(0).getTextContent();
}
}
```

2. Incluye aquí captura/s de pantalla que demuestren la correcta ejecución del código (Y DONDE SEA VISIBLE LA DOCUMENTACIÓN DEL PROYECTO CON JAVADOC Y EL AUTOR DE LA ACTIVIDAD):

```

1  import java.io.File;
2  import java.io.IOException;
3  import javax.xml.parsers.DocumentBuilder;
4  import javax.xml.parsers.DocumentBuilderFactory;
5  import javax.xml.parsers.ParserConfigurationException;
6  import org.w3c.dom.Document;
7  import org.w3c.dom.Element;
8  import org.w3c.dom.NodeList;
9  import org.xml.sax.SAXException;
10
11 //
12 // Author: Juan Gabriel Galarza Clares
13 //
14
15 public class LeerLogs {
16
17     static final String RUTA = "logs.xml"; //usage
18
19     public static void main(String[] args) {
20         try {
21             // ---- PASO 1: cargar el XML en memoria ----
22             // a) Crear la fábrica (DocumentBuilderFactory) y el constructor (DocumentBuilder).
23             DocumentBuilderFactory fabrica = DocumentBuilderFactory.newInstance();
24             DocumentBuilder analizador = fabrica.newDocumentBuilder();
25
26             // b) Leer el fichero y convertirlo en un árbol DOM (Document).
27             Document doc = analizador.parse(new File(RUTA));
28             doc.getDocumentElement().normalize();
29
30             // c) Obtener el elemento raíz y mostrar su nombre.
31             System.out.println("Raiz: " + doc.getDocumentElement().getNodeName());
32         } catch (Exception e) {
33             e.printStackTrace();
34         }
35     }
36 }
  
```

Run Console Output:

```

1  Raiz: logs
2  INFO (sesión iniciada correctamente) (Antonio)
3  ERROR (Intento de acceso fallido) (Ana)
  
```

3. Describe aquí las clases, métodos y variables principales utilizadas en el programa:

DESCRIPCIÓN DE MÉTODOS, CLASES Y VARIABLES

1. Clases Principales

- **LeerLogs:** Es la clase principal de nuestro programa. Contiene todo el código para abrir el archivo XML, leerlo y mostrarlo por pantalla.
- **DocumentBuilder:** Es el "lector" o analizador. Se encarga de coger el archivo de texto XML y transformarlo en una estructura que Java pueda entender.
- **Document:** Representa el archivo XML completo cargado en la memoria del ordenador (como el documento entero listo para ser consultado).
- **Element:** Representa una etiqueta XML concreta (por ejemplo, la etiqueta <log>), lo que permite leer lo que tiene dentro o sus atributos.
- **NodeList:** Es una lista de etiquetas. Sirve para guardar varios elementos juntos (por ejemplo, todos los <log> del archivo).

2. Métodos Principales

- **main:** El punto de partida. Es el método que se ejecuta al pulsar "Play" y se encarga de coordinar todo el proceso paso a paso.
- **obtenerTexto:** Una función de ayuda creada por nosotros. Busca una etiqueta hija y saca su texto; si la etiqueta no existe, devuelve "(sin dato)" para que el programa no se rompa por un error.
- **parse:** Ordena al lector que abra el archivo y construya el documento en memoria.
- **getElementsByTagName:** Busca y devuelve todas las etiquetas que tengan un nombre concreto (como buscar todos los "log").



- **getAttribute:** Lee el valor de un atributo dentro de una etiqueta (por ejemplo, el texto dentro de id="...").

3. Variables Principales

- **RUTA:** Guarda el nombre y ubicación del archivo que vamos a leer ("logs.xml"). No cambia de valor.
- **doc:** Guarda en memoria todo el archivo XML ya procesado y listo para usar.
- **logs:** La lista que contiene todas las etiquetas <log> encontradas en el archivo.
- **u:** Representa el <log> individual que estamos leyendo en cada vuelta del bucle.
- **id, nivel, mensaje, usuario:** Textos donde guardamos temporalmente los datos de cada log para poder imprimirlos por pantalla.